SAVEETHA SCHOOL OF ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

COMPUTER SCIENCE AND ENGINEERING

CSA47 - DEEP LEARNING

ASSIGNMENT 2

CO2 – DEEP FEED-FORWARD AND CONVOLUTIONAL NETWORK DESIGN

Name:	Ch. Tharunika Reddy	Register No.:	192324307
Course:	CSA47 – Deep Learning	Nature:	Individual

Duration: 1 Week Total Marks: 50

CO2: Apply architectural design principles and optimization techniques to construct and train deep feedforward and convolutional neural networks.

Topics: Deep Feed-Forward Networks, Gradient-Based Learning, Back-Propagation, Regularization and Convolutional Networks.

Purpose
Design, train and analyze neural-network architectures rather than merely reproduce a standard model.

Scenario: Smart Agriculture and Crop Disease Detection
An agricultural research centre wants to predict crop yield using rainfall, temperature, soil moisture and fertilizer quantity. It also plans to classify healthy and diseased crop leaves using images. You are required to design, train, optimize and evaluate suitable deep neural network architectures.

TASK 1 – Deep Feed-Forward Network Architecture | 5 Marks
An agricultural centre wants to predict crop yield using rainfall, temperature, soil moisture and fertilizer quantity. A feed-forward network has two hidden layers containing 8 and 4 neurons.

a) Design the network by identifying the input, hidden and output layers and the number of neurons in each layer.

b) Specify the dimensions of the weight matrices and bias vectors.

c) Explain how inputs flow through the network to produce a crop-yield prediction.

Evidence Expected: Network architecture, neuron counts, weight and bias dimensions, mathematical representation and forward propagation.

TASK 2 – Forward Propagation and Activation Functions | 2 Marks
A neuron receives normalized inputs 0.6, 0.8 and 0.5. Its weights are 0.4, 0.3 and 0.2, and its bias is 0.1.

a) Calculate the weighted sum received by the neuron.

b) Apply ReLU and interpret the output.

Evidence Expected: Weighted-sum calculation, activation-function application and interpretation.

TASK 3 – Gradient-Based Learning and Loss Calculation | 6 Marks
A crop-yield network predicts 42 units when the actual yield is 50 units.

a) Explain why a loss function is required and select a suitable loss function for regression.

b) Calculate the squared error loss using $L = (y - \hat{y})^2$.

c) Explain what the loss value indicates about the prediction.

d) Describe how gradient-based learning uses the loss to update network parameters.

Evidence Expected: Suitable loss selection, numerical calculation, interpretation and explanation of gradient-based optimization.

TASK 4 – Gradient Descent and Weight Update | 4 Marks
During training, a network weight is 0.5, its loss gradient is 0.8, and the learning rate is 0.1.

a) Calculate the updated weight using the gradient descent rule.

b) Explain why the update is in the opposite direction to the gradient.

c) Discuss the effects of an extremely small or large learning rate.

Evidence Expected: Gradient descent formula, numerical update, update-direction explanation and learning-rate analysis.

TASK 5 – Back-Propagation Through Multiple Layers | 3 Marks
A crop-yield prediction network contains an input layer, two hidden layers and an output layer. Its prediction differs substantially from the actual yield.

a) Explain how backpropagation determines the contribution of weights to the prediction error.

b) Explain the role of the chain rule in calculating gradients across layers.

c) Describe how gradients are used to update weights.

Evidence Expected: Backpropagation process, chain-rule explanation, gradient flow and weight updates.

TASK 6 – Activation Functions and Vanishing/Exploding Gradients | 3 Marks
A deep network for crop analysis learns slowly because gradients reaching early layers become extremely small.

a) Explain how activation functions can contribute to vanishing gradients.

b) Compare sigmoid, tanh and ReLU for hidden layers.

c) Explain how activation-function choice affects deep-network training.

Evidence Expected: Activation-function comparison, gradient behaviour and impact on training.

TASK 7 – Back-Propagation and Optimization Strategy | 4 Marks
In one experiment, the training loss decreases slowly. In another, it decreases quickly but fluctuates considerably.

a) Explain how the learning rate affects optimization.

b) Explain how backpropagation and gradient descent work together in one iteration.

c) Suggest changes to improve convergence in both experiments.

d) Explain why the loss curve should be monitored.

Evidence Expected: Relationship between backpropagation and optimization, learning-rate analysis, suitable adjustments and loss-curve interpretation.

TASK 8 – Regularization and Overfitting Control | 4 Marks
A crop-yield network achieves 99% training accuracy on a crop-category task but only 78% validation accuracy.

a) Identify the likely learning problem and explain the evidence.

b) Explain how L1 and L2 regularization control model complexity.

c) Write a regularized loss function using an L2 penalty and explain the regularization parameter.

d) Explain how dropout may improve generalization.

Evidence Expected: Identification of overfitting, regularization explanation, loss formulation and dropout-based solution.

TASK 9 – Convolution Operation and Feature Extraction | 3 Marks
An agricultural centre uses leaf images to identify visual signs of disease, including spots, edges and texture.

a) Explain why convolutional layers are suitable for image data.

b) Demonstrate one convolution operation using a small image matrix and filter, and interpret the result.

c) Explain the roles of filters, stride and padding.

Evidence Expected: Convolution-based feature extraction, numerical example and interpretation of filter, stride and padding.

TASK 10 – CNN Architecture and Pooling | 4 Marks

A CNN is proposed to classify leaf images using the architecture: Input Image → Convolution → ReLU → Pooling → Convolution → ReLU → Pooling → Fully Connected Layer → Output.

a) Explain the function of each major component.

b) Explain how pooling reduces feature-map dimensions.

c) Discuss how filter size, stride and number of filters affect feature maps.

d) Explain how multiple convolutional layers learn increasingly complex features.

Evidence Expected: CNN component explanation, pooling operation, feature-map dimensions and hierarchical feature extraction.

TASK 11 – Comparative Evaluation of Feed-Forward and CNN Models | 4 Marks

The centre considers Model A, a feed-forward network using numerical weather and soil features, and Model B, a CNN using crop-leaf images.

a) Explain why Model A is appropriate for structured numerical data.

b) Explain why Model B is suitable for images.

c) Compare the models in input representation, feature extraction, parameter organization and computational requirements.

d) Recommend a model for each task and justify the choice.

Evidence Expected: Technical comparison, relationship between data type and architecture, computational comparison and justified model selection.

TASK 12 – Design Challenge: Optimize and Redesign the Deep Learning System | 8 Marks

Changed Requirement

The agricultural centre initially developed a feed-forward network using 10,000 crop records. It must now process 500,000 crop records and 100,000 leaf images. The existing model has high training accuracy, lower validation performance and long training time. Decide whether to retain, modify or redesign the system.

A. Impact Analysis | 2 Marks

a) Explain how the larger numerical dataset and the addition of leaf images affect computational and architectural requirements.

b) Identify the limitations of applying a feed-forward architecture directly to image data.

B. Evaluation of the Existing Design | 2 Marks

a) Evaluate the existing model using its training and validation performance.

b) Explain whether simply increasing network depth and neuron count is necessarily the best solution.

C. Adaptation or Redesign | 2 Marks

a) Propose a feed-forward network for numerical crop data and a CNN for leaf images.

b) Recommend optimization and regularization methods such as Adam or SGD, learning-rate adjustment, dropout, L2 regularization, batch normalization and suitable activation functions.

D. Evidence-Based Defence | 2 Marks

a) Design an experiment comparing the original and redesigned systems using loss, validation performance, suitable classification/regression metrics and training time.

b) Justify whether the original system should be retained, modified or redesigned based on the expected evidence.

Additional design considerations: Compare SGD, mini-batch SGD, momentum, Adam and RMSProp; propose a learning-rate schedule; design label-preserving image augmentation; and compare LeNet, AlexNet, ResNet and EfficientNet for the available image size and computational budget.

Evidence Expected: Analysis of data volume, revised architectures, justified optimization and regularization choices, image augmentation plan and evidence-based evaluation.

Answers

TASK 1 – Deep Feed-Forward Network Architecture (5 Marks)

a) Network design

The network predicts one continuous value (crop yield) from four inputs, so it is a regression network.

- Input layer: 4 neurons (rainfall, temperature, soil moisture, fertilizer quantity)
- Hidden layer 1: 8 neurons (ReLU)
- Hidden layer 2: 4 neurons (ReLU)
- Output layer: 1 neuron (linear activation, because yield is a real number)

Architecture: 4 → 8 → 4 → 1

b) Weight and bias dimensions

For a layer with n_in inputs and n_out neurons, W has shape (n_out × n_in) and b has shape (n_out × 1).

Layer	Weight matrix	Bias vector	Parameters
Input → Hidden 1	W1: 8 × 4	b1: 8 × 1	32 + 8 = 40
Hidden 1 → Hidden 2	W2: 4 × 8	b2: 4 × 1	32 + 4 = 36
Hidden 2 → Output	W3: 1 × 4	b3: 1 × 1	4 + 1 = 5
Total			81

c) Forward propagation

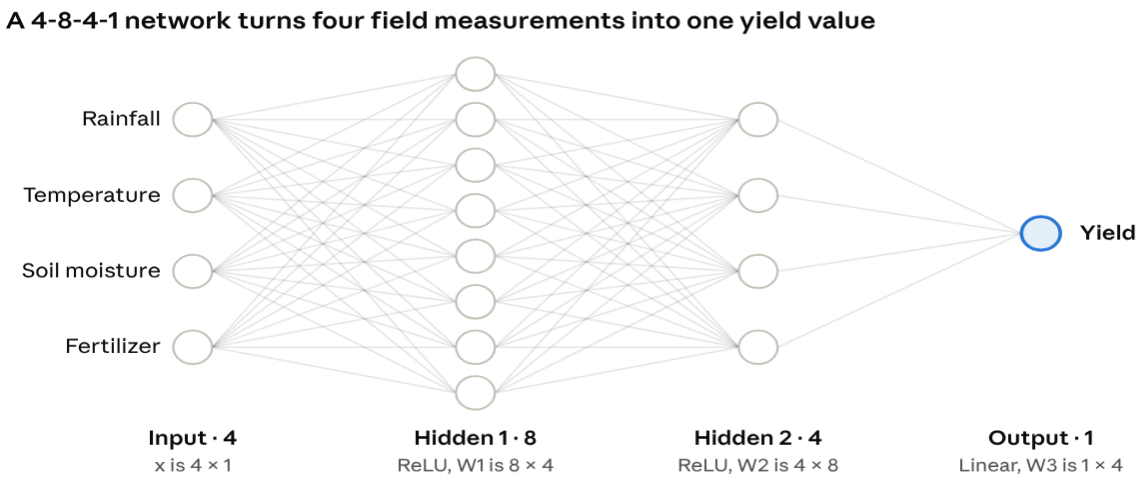
Let x = [rainfall, temperature, soil moisture, fertilizer]^T be a 4 × 1 vector.

- $z1 = W1 \cdot x + b1$ (8 × 1), $a1 = \text{ReLU}(z1)$
- $z2 = W2 \cdot a1 + b2$ (4 × 1), $a2 = \text{ReLU}(z2)$
- $\hat{y} = W3 \cdot a2 + b3$ (1 × 1), the predicted yield

Each layer computes a weighted sum, adds a bias and applies an activation. The output of one layer becomes the input of the next, flowing only forward. The non-linear ReLU activations let the network learn non-linear relationships, such as yield rising with fertilizer only up to a point.

Result: A 4-8-4-1 feed-forward network with 81 trainable parameters was designed.

Output – Network architecture



TASK 2 – Forward Propagation and Activation Functions (2 Marks)

Inputs x = [0.6, 0.8, 0.5], weights w = [0.4, 0.3, 0.2], bias b = 0.1.

a) Weighted sum

$z = (0.6 \times 0.4) + (0.8 \times 0.3) + (0.5 \times 0.2) + 0.1$

$z = 0.24 + 0.24 + 0.10 + 0.10 =$ **0.68**

b) ReLU

$\text{ReLU}(z) = \max(0, z) = \max(0, 0.68) =$ **0.68**

Interpretation: z is positive, so the neuron is active and passes its value on unchanged. Had z been negative, the output would be 0 and the neuron would stay silent for that input.

Result: Weighted sum = 0.68 and neuron output after ReLU = 0.68.

TASK 3 – Gradient-Based Learning and Loss Calculation (6 Marks)

a) Why a loss function is needed

A loss function turns the gap between prediction and truth into a single number. Without it the network has no measure of how wrong it is and no gradient to follow when improving. For regression, **Mean Squared Error (MSE)** is a suitable choice: it is smooth and differentiable, and it penalizes large errors more heavily. Mean Absolute Error or Huber loss are alternatives when outliers are present.

b) Squared error

$y = 50$, $\hat{y} = 42$

$L = (y - \hat{y})^2 = (50 - 42)^2 = 8^2 =$ **64**

c) Interpretation

The prediction is 8 units below the actual yield (an underestimate of 16%). A loss of 64 is large relative to the scale of the data, so the network needs to adjust its parameters. A loss of 0 would mean a perfect prediction.

d) Gradient-based learning

1. Forward pass: compute $\hat{y}$ and the loss L.
2. Compute the gradient $\partial L/\partial\theta$ of the loss with respect to every weight and bias. For the output here, $\partial L/\partial\hat{y} = -2(y - \hat{y}) = -16$, which says that increasing $\hat{y}$ would reduce the loss.
3. Update each parameter in the direction that reduces loss: $\theta \leftarrow \theta - \eta \cdot \partial L/\partial\theta$.
4. Repeat over many iterations until the loss stops decreasing.

Result: MSE selected, loss = 64, and the gradient of the loss is used to update parameters.

TASK 4 – Gradient Descent and Weight Update (4 Marks)

$w = 0.5$, $\partial L/\partial w = 0.8$, $\eta = 0.1$.

a) Update

$w_new = w - \eta \cdot \partial L/\partial w = 0.5 - (0.1 \times 0.8) = 0.5 - 0.08 =$ **0.42**

b) Why the opposite direction

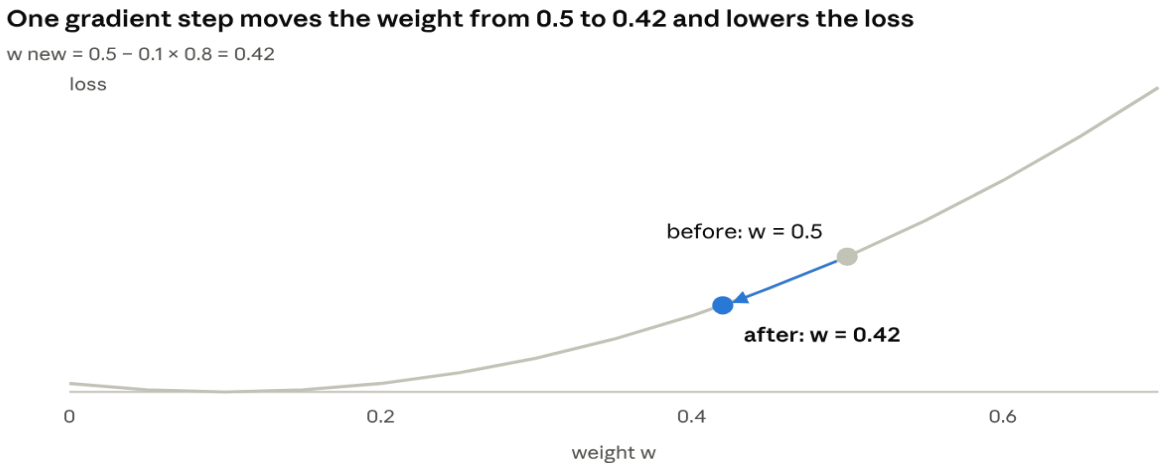
The gradient points in the direction in which the loss increases fastest. To reduce the loss we must move against it. Here the gradient is positive, so increasing w would increase the loss; decreasing w to 0.42 lowers it.

c) Effect of learning rate

- Extremely small η: the updates are tiny, so training is very slow and can stall on flat regions or in poor local minima.
- Extremely large η: the updates overshoot the minimum, so the loss oscillates or even diverges.
- A moderate η (often with a decaying schedule) gives fast and stable convergence.

Result: Updated weight = 0.42.

Output – Gradient descent on the loss curve



TASK 5 – Back-Propagation Through Multiple Layers (3 Marks)

a) Contribution of weights to the error

Backpropagation first runs a forward pass to get the prediction and loss. It then travels backward from the output layer, calculating for every weight how much a small change in it would change the loss ($\partial L/\partial w$). Weights with large gradients contributed more to the error.

b) Role of the chain rule

The loss depends on early-layer weights only through the later layers, so gradients are built by multiplying local derivatives. For a weight in the first hidden layer:

$\partial L/\partial W1 = \partial L/\partial \hat{y} \cdot \partial \hat{y}/\partial a2 \cdot \partial a2/\partial z2 \cdot \partial z2/\partial a1 \cdot \partial a1/\partial z1 \cdot \partial z1/\partial W1$

Each layer reuses the gradient already computed for the layer after it, which makes the computation efficient.

c) Weight update

Once all gradients are known, each weight and bias is updated with $W \leftarrow W - \eta \cdot \partial L/\partial W$. The next forward pass then gives a lower error, and the cycle repeats.

Result: Backpropagation uses the chain rule to propagate error gradients from output to input and update all weights.

TASK 6 – Activation Functions and Vanishing/Exploding Gradients (3 Marks)

a) How activations cause vanishing gradients

In backpropagation, gradients are multiplied by the activation derivative at every layer. Sigmoid has a maximum derivative of 0.25 and tanh of 1, and both are close to 0 when saturated. Multiplying many numbers smaller than 1 across deep layers makes the gradient shrink exponentially, so early layers barely learn. Conversely, derivatives or weights larger than 1 can make gradients explode.

b) Comparison

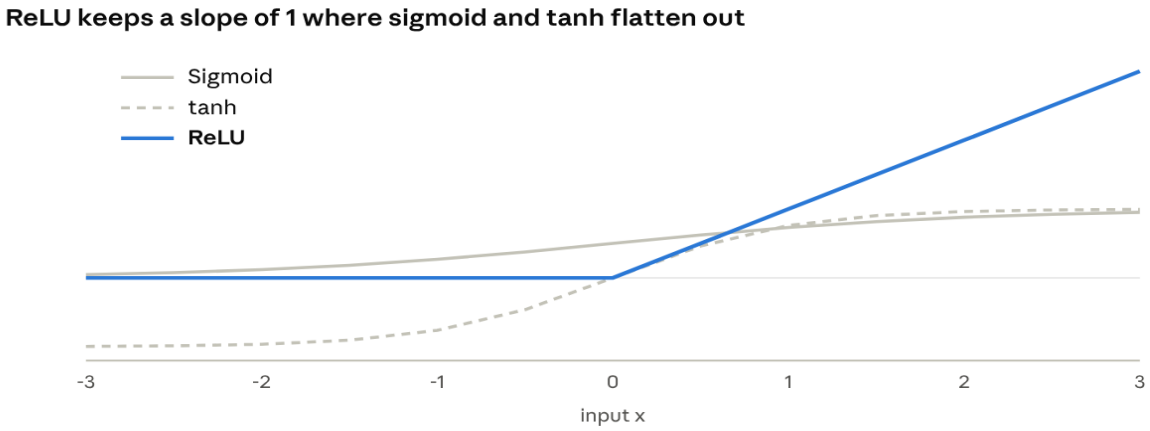
Activation	Range	Gradient behaviour	Use in hidden layers
Sigmoid	(0, 1)	Max 0.25, saturates, not zero-centred	Poor for deep networks
tanh	(−1, 1)	Max 1, zero-centred but still saturates	Better than sigmoid, still risky
ReLU	[0, ∞)	Gradient 1 for positive inputs, cheap	Default choice; can have "dead" neurons

c) Effect on training

ReLU (and variants such as Leaky ReLU) keeps the gradient at 1 for active units, so it reaches early layers and training is faster. Combined with careful weight initialization (He), batch normalization, residual connections and gradient clipping (for explosion), it makes deep networks trainable.

Result: ReLU-type activations are recommended for the deep crop-analysis network.

Output – Activation functions and their derivatives



Sigmoid slope peaks at 0.25 and tanh at 1, then both flatten. ReLU slope stays 1 for x above 0.

TASK 7 – Back-Propagation and Optimization Strategy (4 Marks)

a) Learning rate and optimization

The learning rate sets the step size. A low rate gives smooth but slow loss reduction (experiment 1). A high rate gives fast initial progress but overshoots, so the loss fluctuates (experiment 2).

b) Backpropagation with gradient descent in one iteration

1. Forward pass on a (mini-)batch to compute predictions and loss.
2. Backpropagation computes gradients of the loss for all parameters.
3. Gradient descent uses those gradients to update parameters: $\theta \leftarrow \theta - \eta \nabla L$.

Backpropagation supplies the gradients; gradient descent decides how to use them.

c) Improving convergence

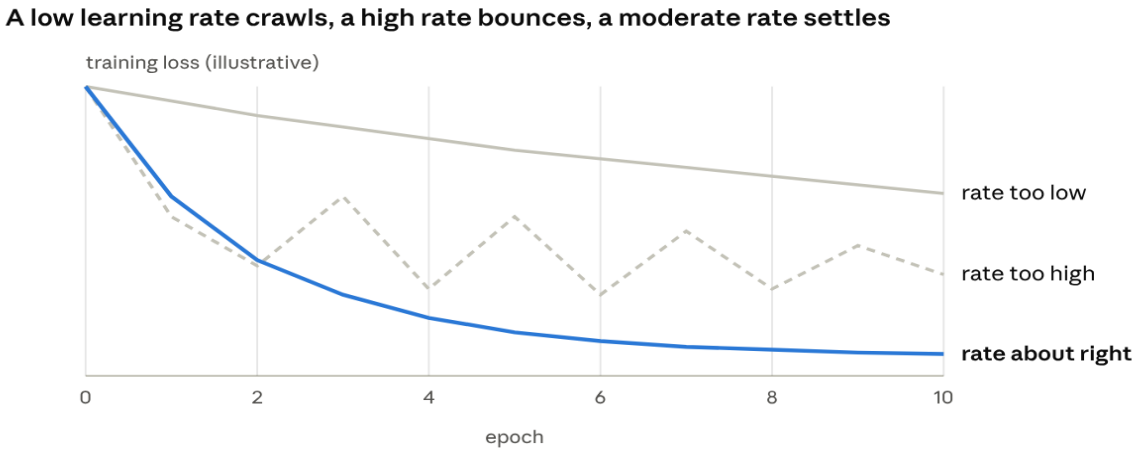
- Experiment 1 (slow): increase the learning rate, use momentum or Adam, normalize inputs, use batch normalization and a better initialization.
- Experiment 2 (fluctuating): reduce the learning rate, apply a decay schedule, increase batch size, use momentum or gradient clipping.

d) Why monitor the loss curve

It shows whether training is converging, diverging or stalled, reveals a poor learning rate, and, when plotted alongside validation loss, shows overfitting early so that early stopping can be applied.

Result: Learning rate adjustments and adaptive optimizers address both behaviours.

Output – Loss curves for different learning rates



TASK 8 – Regularization and Overfitting Control (4 Marks)

a) Problem identified

Overfitting. Training accuracy is 99% while validation accuracy is only 78%, a 21-point gap. The model has memorized the training data, including noise, and does not generalize.

b) L1 and L2 regularization

- L1 adds $\lambda\Sigma|w|$ to the loss. It pushes many weights to exactly zero, producing sparse models and a form of feature selection.
- L2 adds $\lambda\Sigma w^2$ to the loss. It shrinks all weights toward zero (weight decay) without making them exactly zero, giving smoother functions.

Both penalize large weights, which limits the effective complexity of the model.

c) L2-regularized loss

$$L_total = (1/N)\ \Sigma\ (y_i - \hat{y}_i)^2 + \lambda\ \Sigma\ w_j^2$$

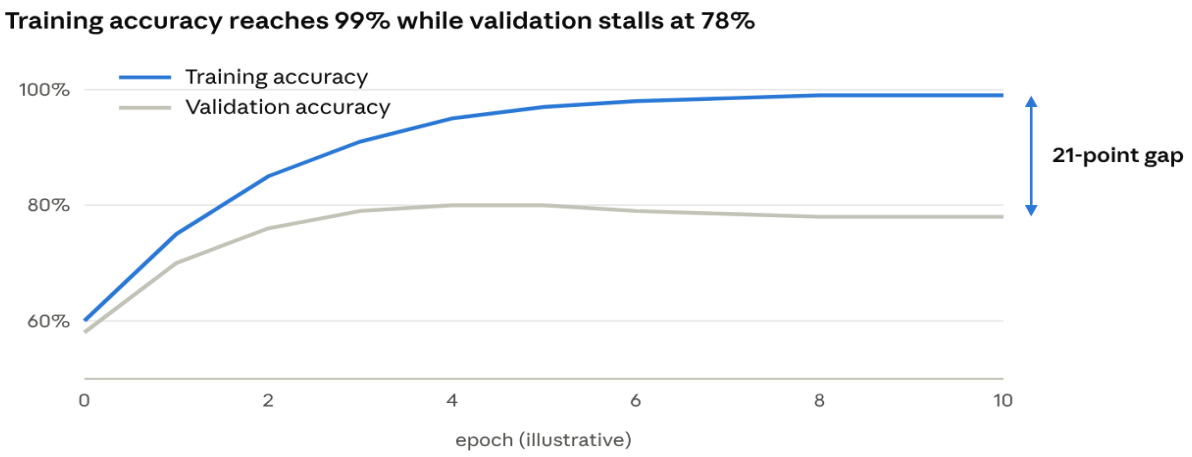
λ (regularization parameter) controls the strength of the penalty. $\lambda = 0$ means no regularization and risks overfitting; a very large λ forces weights towards zero and causes underfitting. It is usually tuned using validation data (for example 10^{-4} to 10^{-2}).

d) Dropout

During training, each neuron is randomly switched off with probability p (for example 0.3 to 0.5). The network cannot rely on specific neurons, so it learns redundant, robust features, behaving like an ensemble of many sub-networks. At test time all neurons are used with scaled outputs.

Result: Overfitting was identified and can be reduced using L1/L2, dropout and early stopping.

Output – Training vs validation accuracy



TASK 9 – Convolution Operation and Feature Extraction (3 Marks)

a) Why convolutional layers suit images

- Local connectivity: each filter looks at a small patch, which suits local patterns such as spots and edges.
- Weight sharing: the same filter slides over the whole image, so far fewer parameters are needed than in a fully connected layer.
- Translation invariance: a lesion is detected wherever it appears on the leaf.
- Hierarchical features: edges combine into textures, then into disease patterns.

b) Convolution example

Image (4 × 4):

```
1 0 2 3
4 1 0 2
3 2 1 0
0 1 3 2
```

Vertical-edge filter (3 × 3):

```
1 0 -1
1 0 -1
1 0 -1
```

First position (top-left 3 × 3 window):

$(1\times1 + 0\times0 + 2\times(-1)) + (4\times1 + 1\times0 + 0\times(-1)) + (3\times1 + 2\times0 + 1\times(-1)) = -1 + 4 + 2 = \mathbf{5}$

Sliding with stride 1 gives the 2 × 2 feature map:

```
5                                                                -2
3  0
```

Interpretation: a large positive value (5) means a strong left-bright to right-dark vertical edge in that patch, a negative value means the opposite edge direction, and 0 means no vertical edge. On a leaf, such responses highlight lesion boundaries.

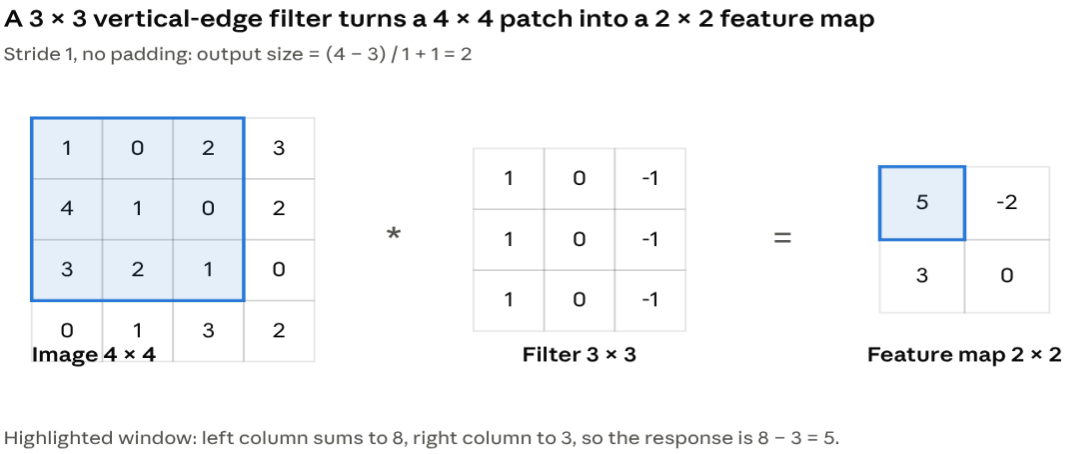
c) Filters, stride and padding

- Filter (kernel): a small weight matrix that detects one feature; the weights are learned.
- Stride: the step size of the slide; larger stride gives a smaller output and less computation.
- Padding: zeros added around the border; "same" padding keeps the output size and preserves edge information.

Output size = $(n - f + 2p) / s + 1 = (4 - 3 + 0)/1 + 1 = 2$.

Result: The convolution produced a 2 × 2 feature map [[5, −2], [3, 0]].

Output – Convolution and feature map



TASK 10 – CNN Architecture and Pooling (4 Marks)

a) Components

- Input image: raw pixels of the leaf (for example 64 × 64 × 3).
- Convolution: filters extract local features such as edges and spots.
- ReLU: adds non-linearity and avoids vanishing gradients.
- Pooling: down-samples the feature maps, keeping the strongest responses.
- Fully connected layer: combines all extracted features to make a decision.
- Output: softmax (or sigmoid) gives the probability of healthy versus diseased.

b) How pooling reduces dimensions

Max pooling with a 2 × 2 window and stride 2 keeps the largest value in each window. A 4 × 4 map becomes 2 × 2, halving height and width and leaving 25% of the values. This lowers computation and parameters and gives small translation invariance.

Example: [[1,3],[2,4]] → 4.

c) Effect of filter size, stride and number of filters

- Filter size: small filters (3 × 3) capture fine detail with fewer parameters; large filters see wider context but cost more and shrink the map more.
- Stride: larger stride produces smaller maps.
- Number of filters: sets the depth of the output (channels). More filters detect more features but increase parameters and memory.

Worked dimensions with 3 × 3 filters and "same" padding:

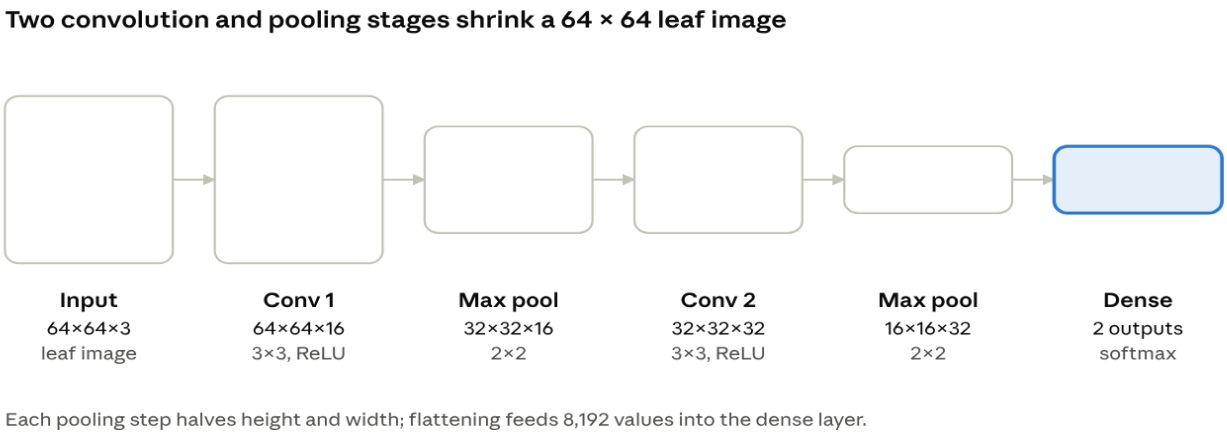
- Input 64 × 64 × 3
- Conv1 (16 filters): 64 × 64 × 16, parameters (3·3·3+1)·16 = 448
- Pool 2 × 2: 32 × 32 × 16
- Conv2 (32 filters): 32 × 32 × 32, parameters (3·3·16+1)·32 = 4,640
- Pool 2 × 2: 16 × 16 × 32 = 8,192 values after flattening
- Fully connected to 2 outputs: 8,192 × 2 + 2 = 16,386 parameters

d) Hierarchical features

Early layers learn simple patterns (edges, colour blobs). Later layers combine them into textures and shapes (spots, vein patterns), and the deepest layers represent disease-specific patterns. Pooling widens each neuron's receptive field, enabling this build-up.

Result: The CNN converts a leaf image into a healthy/diseased prediction through hierarchical feature extraction.

Output – CNN architecture



TASK 11 – Comparative Evaluation of Feed-Forward and CNN Models (4 Marks)

a) Why Model A suits structured numerical data

Rainfall, temperature, soil moisture and fertilizer are a small set of independent numerical features with no spatial arrangement. A fully connected network can model their interactions directly and is light to train.

b) Why Model B suits images

Images have spatial structure. A CNN exploits local patterns with shared filters, giving translation invariance and far fewer parameters than a dense network on raw pixels.

c) Comparison

Aspect	Model A (feed-forward)	Model B (CNN)
Input	Vector of 4 features	Image tensor, e.g. 224 × 224 × 3
Feature extraction	Manual features, learned combinations	Learned automatically (edges → textures → shapes)
Parameters	Dense matrices, every input connected to every neuron	Small shared filters reused across the image
Computation	Low	High (GPU recommended)

d) Recommendation

- Yield prediction: Model A, because the data is tabular and numeric, and a small network is fast, accurate and easy to train.
- Leaf disease classification: Model B, because only a CNN can learn the visual patterns of disease from pixels efficiently.

Result: Model A for yield prediction and Model B for disease classification.

TASK 12 – Design Challenge: Optimize and Redesign the Deep Learning System (8 Marks)

A. Impact Analysis (2 Marks)

a) Effect of larger data and added images

- Numerical data grows from 10,000 to 500,000 records: more computation per epoch, so mini-batch training is needed, and there is more data to support a deeper or wider model with less overfitting risk.
- 100,000 leaf images are a different data type. A 224 × 224 × 3 image has 150,528 values, and the image set holds about 15 billion values, which demands GPUs, efficient data loading, and convolutional architectures.
- The system now needs two input pipelines and two models rather than one.

b) Limits of a feed-forward network on images

- Flattening destroys spatial structure.
- Parameter explosion: 150,528 inputs × 100 hidden neurons is about 15 million weights in the first layer alone.
- No translation invariance, so a lesion in a new position looks like a new pattern.
- High overfitting risk and heavy memory and training cost.

B. Evaluation of the Existing Design (2 Marks)

a) Evaluation

High training accuracy with lower validation performance shows overfitting (high variance). Long training time shows an inefficient setup for the current data volume. The model also cannot process images.

b) Is more depth and neurons the answer?

No. More capacity increases variance and training time and makes overfitting worse unless there is enough data and regularization. The gap indicates a generalization problem, which is better handled with regularization, better optimization, early stopping and the right architecture for each data type.

C. Adaptation or Redesign (2 Marks)

a) Proposed architectures

- Numerical crop data: feed-forward 4 → 64 → 32 → 16 → 1 with ReLU, batch normalization, dropout 0.2–0.3, and a linear output.
- Leaf images: CNN. Start with a lightweight pretrained model such as EfficientNet-B0 or ResNet-18/50 (transfer learning), replacing the final layer with a 2-class softmax output. Train from scratch only a small custom CNN if no pretrained weights are available.

b) Optimization and regularization

- Optimizer: Adam (learning rate 1e-3 for the feed-forward net, 1e-4 for fine-tuning the CNN); SGD with momentum 0.9 is a strong alternative for the CNN.
- Learning-rate schedule: warm-up for a few epochs, then cosine decay (or reduce on plateau by 0.1).
- Regularization: L2 weight decay (1e-4), dropout, batch normalization, early stopping on validation loss.

- Activations: ReLU or Leaky ReLU in hidden layers, linear output for regression, softmax or sigmoid for classification.
- Mini-batches of 64–256 for tabular data and 32–64 for images.

Optimizer comparison

- SGD: simple but slow and noisy.
- Mini-batch SGD: balances speed and stability, and is the practical standard.
- Momentum: accumulates past gradients, which smooths oscillations and speeds up convergence.
- RMSProp: scales the step by recent gradient magnitude, which suits non-stationary problems.
- Adam: combines momentum and RMSProp, converges fast with little tuning.

Image augmentation (label-preserving)

Random flips, rotation up to 30°, small zoom and crop, brightness and contrast jitter, and slight colour shifts. Avoid strong hue changes or heavy cropping that could remove or alter the disease symptoms.

Architecture comparison for leaf images

- LeNet (about 60 thousand parameters): too small for realistic leaf textures; fine for 32 × 32 inputs.
- AlexNet (about 61 million): heavy, mostly in its dense layers, and outdated.
- ResNet-50 (about 25 million): robust residual connections that ease deep training; accurate but costlier.
- EfficientNet-B0 (about 5 million): best accuracy per parameter and per FLOP. Recommended for the available image size and computational budget.

D. Evidence-Based Defence (2 Marks)

a) Experiment

Use the same split (70% train, 15% validation, 15% test) for both systems.

- Original: one feed-forward network, with images flattened.
- Redesigned: feed-forward for numerical data plus EfficientNet-B0 for images.
- Record per epoch: training loss, validation loss and training time; also memory use.
- Regression metrics: MSE, RMSE, MAE and R^2.
- Classification metrics: accuracy, precision, recall, F1-score, ROC-AUC and the confusion matrix.
- Repeat with several random seeds, and perform ablations (with and without dropout, L2 and augmentation).

b) Decision: redesign

The original system should be **redesigned**, not simply retained or modified. It cannot handle images effectively, it overfits, and it is slow at the new scale. The redesigned system is expected to show a smaller train–validation gap, lower validation error for yield, higher F1 and recall for disease detection, and shorter training time per epoch thanks to mini-batching and transfer learning.

Final Result: Replace the single feed-forward model with a feed-forward network for yield prediction and a transfer-learned CNN (EfficientNet-B0) for leaf disease classification, trained with Adam, a cosine schedule, dropout, L2 regularization, batch normalization and augmentation.

Output – Proposed redesigned system

